Complete codebase review and rewrite roadmap

Prepared 24 September 2026 from the saved baseline, implementation evidence, and current service/API directory inventory. This is an execution plan, not a claim that the remaining review has been performed.

Objective and present position

Review the entire maintained application, make an evidenced retain/consolidate/replace/remove decision for every area, implement the justified changes, and verify the integrated product. Preserve supported capabilities and contracts. Prioritize correctness/security, clarity, measured performance, then size. A reviewed and justified retained module counts toward completion; an untouched, unaudited module does not.

The original measured application contains 1,602 files and 383,365 physical lines, with 102 direct production dependencies. The scan identified 62 exact repeated-body groups. Seven of eight original import-cycle groups disappear during TypeScript transpilation; one accounting/inventory group retains an emitted-code cycle. That alone does not establish a runtime defect.

Already implemented: cash-history cache isolation, grouped summary reads, and consolidation of 106 preflight wrappers. The net attributable application reduction is 1,745 lines. The six focused suites passed 1,019 tests; 11 hook tests passed again after the last type fix; the final full typecheck passed. Those results apply to the recorded file hashes, not to arbitrary future changes.

Open foundations: the isolated build timed out after 15 minutes, local app probes timed out, real-database performance and authenticated browser checks are absent, and the inventory gate reports 20 active findings outside the edited files. The full rewrite remains incomplete.

Execution order

Stage	Work	Required output and completion gate
00 — Establish reliable verification	Diagnose the build timeout, obtain a responsive local/test app and dedicated test database, and triage all 20 inventory findings. Finish verification of the implemented slice.	Reproducible build/start/test commands; authenticated browser evidence; database baseline; every finding assigned an evidence-backed disposition. An unresolved required gate remains explicitly blocking.
01 — Inventory the whole product	Map every first-party area and every supported workflow, including dynamic entry points and excluded/legacy code.	Complete scope/workflow registers with owners, contracts, dependencies, risks, and tests. No unassigned application areas.
02 — Freeze architecture and targets	Define domain ownership, contract boundaries, dependency direction, and measured reduction/performance targets.	Architecture decisions, target ledger, and ordered work packages. Every proposed change has a reason, consumer inventory, verification method, and rollback path.
03 — Simplify shared foundations	Review identity/RBAC/tenancy, entitlements, data access, query caching, common UI, errors, evidence, events, and provider contracts.	Shared controls pass positive and negative tests; chosen duplication is removed without creating a generic framework for unrelated domains.

Stage	Work	Required output and completion gate
04 — Inventory, POS, and offline workflows	Review item/location/stock ownership, valuation, counts, transfers, adjustments, cash drawers, tenders, receipts, returns, replay, and end-of-day operations.	Stock, cash, posting, idempotency, and replay invariants pass under normal, failure, retry, and concurrent execution.
05 — Commercial and payment workflows	Review procurement/suppliers/AP, customers/order-to-cash/AR, refunds, payments, mobile money, settlement, suspense, and reconciliation.	Orders, receipts, invoices, approvals, money movement, and ledger evidence reconcile across their complete lifecycles.
06 — Accounting, close, and statutory boundaries	Review posting, invalidation, periods, close packs, fiscal documents, country adapters, audit records, and accountant-facing workflows.	Financial tie-outs, closed-period rules, correction/invalidation behavior, and evidence integrity pass. Changes to statutory rules retain dated provenance and qualified review.
07 — HRIS and payroll	Review employee/contract/compensation/time data through calculation, approval, payment, declarations, self-service, and ledger/close integration.	Approved-source snapshots, monetary calculations, privacy, segregation of duties, corrections, and accounting tie-outs pass.
08 — Complete the remaining product areas	Review analytics/BI, dashboards and command centers, onboarding, organization/admin/media, modules/billing/provider boundaries, referrals, communications, AI/agents, signals, and other inventory-discovered modules.	Every remaining supported area has a completed decision and verified workflows; canonical metrics and action authority have explicit owners.
09 — Optimize and remove superseded code	Profile the integrated product, remove demonstrated bottlenecks, consolidate justified repetition, and retire verified obsolete implementations/dependencies.	Comparable before/after measurements meet frozen budgets; consumers have migrated; temporary layers are removed or explicitly accounted for.
10 — Validate the complete result	Run integrated workflows, policy/build/security/recovery checks, browser/accessibility/localization checks, and final scope reconciliation.	Every required completion condition is met, with unresolved requirements preventing a complete/ready claim. Handover is prepared; production deployment is outside this roadmap's execution authorization.

Stages 00 and 01 can advance independently. Domain discovery can continue while an environment issue is investigated. Broad replacement work depends on stages 01–03. Domain implementation follows verified producer/consumer dependencies; the table is the default order, and stage 02 must refine it from the actual workflow map. Security, UI, accessibility, data integrity, and operations checks apply within every stage.

Stage 00: the immediate work queue

1. Establish one reproducible source snapshot containing the accepted working-tree changes. Record hashes, dependencies, environment versions, build output directory, and database identity. Preserve other tasks' work; a checkout of HEAD alone would omit the currently implemented changes.
2. Diagnose the build timeout with build trace/timing, resource measurements, dependency fan-out, and generated-file inclusion checks. Distinguish host contention from application/build-system problems. Increasing the timeout is not itself a correction. Obtain a completed artifact and record its size.
3. Restore a responsive isolated test app. Establish dedicated authorized, denied, cross-tenant, and module-disabled test identities. Keep fixtures in an identified non-production test database. Do not reuse live financial data as disposable fixtures.

4. Create one triage entry per inventory-gate finding: file/line, producer, execution context, violated rule, disposition, owner, and evidence. Fix active stock writers through the authoritative service/transaction path. Preserve historical evidence. Any proposed classification or policy correction needs specific evidence and review; blanket exclusions or automatic baseline refreshes do not satisfy the gate.
5. Exercise cash history in the browser for actor/tenant/permission changes, filtering, pagination, selection, and export preparation. Validate the grouped queries against real PostgreSQL with the same filters and decimal data as the original behavior. Record plans, query time, row counts, endpoint latency, and resource use.
6. Record a fresh verification snapshot. Reuse prior passing evidence where inputs remain identical; rerun checks whose code, dependencies, schema, configuration, or fixtures changed.

Deliverables: `environment-and-gates.md`, `inventory-findings.csv`, browser/database evidence under `evidence/`, and updates to `verification.md`. These are planned artifacts; they are not represented as already completed.

Stage 01: what a complete inventory must cover

Inventory routes, pages, server actions, API endpoints, jobs, webhooks, event/outbox consumers, imports/exports, scheduled work, external providers, feature/module switches, schema/migrations, scripts, CI/deployment configuration, and dependencies. Classify maintained tests, generated/vendor files, historical evidence, and documentation separately.

For each supported workflow record:

- User/business outcome, entry points, actors, permissions, tenant scope, and module requirements.
- Lifecycle states, authoritative service and data owner, public payloads, and downstream consumers.
- Monetary, stock, accounting, privacy, concurrency, idempotency, and audit invariants where applicable.
- UI states, accessibility/localization requirements, errors, retries, recovery, and support procedures.
- Existing tests, missing behavior protection, known defects, performance baseline, and acceptance budgets.

Use `graphify-out/` as a discovery aid and check relationships against current imports/callers. Trace the one emitted accounting/inventory cycle before deciding to retain or change it. Check the 51 files absent from TypeScript root membership for transitive coverage and active use. Treat neither static unused-code output nor a tsconfig exclusion as proof that a feature can be deleted.

Create `scope-register.csv` and `workflow-contracts.md`. Each application area must be mapped to a work package or an evidenced retained/retired disposition. Include legacy or less-visible capabilities discovered outside the initial service list; the named domain waves are not a hidden scope boundary.

Stage 02: architecture and target decisions

Use the existing modular monolith as the starting point. Keep business truth in authoritative domain services, with explicit action/API authorization, transaction boundaries, and provider interfaces. Avoid changing frameworks or introducing services merely to make files shorter.

Review each of the 62 repeated-body groups individually. Identical code may represent shared ownership or intentional separation of different business policies; consolidation requires more than a text match. Prioritize high-risk controls, heavily used slow paths, and repeated authoritative logic before cosmetic reductions.

The target ledger must state baseline, target/budget, dataset, environment, measuring command, variance threshold, and acceptance evidence for:

Measure	Required treatment
Application size	Same counting rules as the original baseline; separate new code, removed code, tests, generated output, archives, and concurrent edits. Include the helpers/dependencies introduced by a replacement.
Complexity	Record repeated logic, executable dependency cycles, authority/ownership duplication, and steps needed to trace representative workflows. Splitting a large file alone is not a simplification result.
Performance	Record p50/p95 latency, throughput, memory, database query count/time/rows, browser responsiveness, bundle sizes, and artifact size where applicable. Fix workload, concurrency, cache state, and repeat count.
Reliability/security	Record critical-workflow coverage, failure-path coverage, verified findings, and required boundary tests. Test count alone is not coverage.

Set numerical targets after stage 00/01 measurements. There is currently no defensible whole-platform reduction percentage or production-latency promise. Define a budget before each replacement; do not adjust it afterward merely to label a result successful.

Deliverables: `target-architecture.md`, `decision-register.md`, `targets.json`, and `work-packages.csv`. Each work package names its owner responsibility, dependencies, exact scope, acceptance criteria, required evidence, and migration/removal conditions.

Domain acceptance requirements

Area	Minimum behavioral protection
Platform authority and SaaS modules	Cross-tenant denial; role/permission changes; invitation and organization boundaries; module activation/deactivation; subscription/provider reconciliation; cache separation; no client authority replacing server checks.
Inventory/POS/offline	Stock conservation and valuation; location/tenant scope; refunds and tender separation; atomic business writes; duplicate replay; interrupted sync; concurrent sale/count/transfer; recovery without double posting.
Procurement/AP and customers/AR	Receipt/invoice/credit relationships; approval authority and segregation of duties; duplicate/concurrent requests; balance and ledger tie-outs; correction and cancellation behavior.
Payments/reconciliation	Verified provider callbacks; duplicate and out-of-order events; reversals; settlement/suspense transitions; redacted evidence; retries without duplicate financial effects. Use provider test facilities or controlled fixtures for execution evidence.
Accounting/close/compliance	Balanced journal entries; source-to-ledger ownership; closed-period protection; invalidation on upstream correction; immutable records; country-pack/version provenance; reconciled close evidence.
HRIS/payroll	Employee/organization scope; approved effective-dated sources; frozen calculation inputs; monetary precision; approvals and correction lineage; payment/declaration evidence; employee self-service privacy.
Analytics/BI/command centers	Canonical KPI owners; consistent filters, timestamps, currency rules, and permissions; freshness/completeness states; tie-out to source evidence; no duplicate business truth in dashboards.
AI/agents	Tool and data boundaries; tenant isolation; untrusted-content handling; required human decisions; denial behavior; evaluation fixtures; bounded runtime/cost; no model output becoming authoritative financial truth.
APIs/events/communications/media	Stable consumer contracts; authentication/signatures; outbox/idempotency behavior; retry and dead-letter recovery; import/export redaction; upload/download access; safe provider failure handling.

Area	Minimum behavioral protection
Product experience and operations	Supported roles complete their workflows on relevant screen sizes; loading/empty/denied/error/recovery states; keyboard/accessibility and localization checks; useful monitoring and support/runbook evidence.

The reviewer responsibilities in `reviewer-coverage.md` apply to every affected package. Record actual evidence, unresolved concerns, or a specific not-applicable reason. Engineering checks do not substitute for required professional statutory review.

The execution loop for every work package

1. Trace current behavior, consumers, and authority boundaries; distinguish defects from intended behavior.
2. Capture the baseline and set acceptance budgets. Add focused characterization/contract tests where evidence is missing.
3. Implement the smallest coherent replacement through its UI, action/API, service, persistence, and integration boundaries as applicable.
4. Verify positive/negative authority cases, business invariants, failure/recovery, and the changed performance boundary.
5. Migrate all consumers. Remove the superseded implementation only after consumer and dynamic-entry-point checks pass. Any temporary adapter needs an owner and an explicit removal condition.
6. Run the relevant integration/policy/build checks, review the diff, and record before/after evidence against exact source hashes.
7. Mark the package verified only when its acceptance criteria pass. Otherwise keep it in implementation, verification, or blocked state and record the concrete unresolved dependency.

For required schema changes, use compatible migration stages, test against an isolated database, reconcile data, and exercise recovery before a migration is considered verified. No destructive reset of business data is a rollback strategy. Production cutover remains separate from this codebase task.

Program tracking and completion

Use one register with: package ID, workflow/domain, owner responsibility, risk, dependencies, status, retain/consolidate/replace/remove decision, baseline hash, acceptance criteria, evidence links, measured deltas, adapters/removal conditions, blockers, and next action.

Allowed progress states: unreviewed, inventoried, characterized, implementing, verifying, verified, blocked. A blocked package must name the missing capability or decision and what resolves it. A retained area reaches verified only after its required review and applicable checks, not merely because no edit was made.

Report progress by verified required workflows and completed work packages, alongside risk and coverage. Do not use deleted lines, file count, or number of passing tests as the sole completion percentage.

The complete review/rewrite is finished only when:

- Every first-party area is accounted for and every required workflow has a verified disposition.
- All required authority, data-integrity, business-invariant, and consumer-contract checks pass.
- A reproducible production build, applicable policy gates, authenticated browser checks, and integrated cross-domain workflows pass.

- Size, complexity, performance, and reliability results are measured consistently; unmet targets are explicitly reconciled rather than hidden.
- Superseded paths and temporary migration layers are removed or explicitly accepted with documented ownership and limits.
- Rollback/recovery evidence exists for affected boundaries, and required statutory review is recorded where rules changed.
- Architecture, operations, support documentation, source hashes, and the final evidence index describe the delivered implementation.

The immediate milestone is **verified completion of stages 00–02**, yielding a reliable execution environment and the full, prioritized implementation backlog. That supplies the basis for a credible schedule; the current source count alone does not.